AIBUILDAI RESEARCH

AIBuildAI Inference Agent: A Recursive Self-Improving Agent for Automatically Building High-Speed LLM Inference Engines

Li Zhang, Ruiyi Zhang, Pengtao Xie*
AIBuildAI Team

Abstract

Large language models (LLMs) are now widely deployed in chat assistants, coding tools and autonomous agents, where generation speed directly determines responsiveness and user experience. Speculative decoding accelerates generation by using a lightweight draft model to quickly propose several tokens, which the target LLM then verifies in a single forward pass. Its speedup therefore depends on how closely the draft model approximates the target LLM, and building such a draft model for each LLM requires substantial expertise and manual effort. To address this, we present the AIBuildAI Inference Agent, a recursive self-improving agent that automatically builds high-speed LLM inference engines by training high-performing EAGLE draft heads, lightweight draft models built on the target LLM's internal features. In each iteration, the agent proposes a draft-head design, spanning architecture, training data, training procedure and draft-tree configuration, and trains the head. Its speedup is then measured by a separate scoring program that the agent cannot access, which prevents reward hacking. The proposed design and its results are then passed to the next iteration, where the agent further improves on them. In addition, the AIBuildAI Inference Agent contains a knowledge base of past experimental findings that reduces the expertise and computation this search requires; the knowledge base is updated after every run, so the agent improves over time. Experiments show that the agent's automatically built draft head makes generation **1.5**× faster than SGLang and **1.8**× faster than vLLM and **6.2**× faster than a HuggingFace generation loop, with unchanged accuracy. Specifically, the results are measured on Qwen2.5-14B-Instruct served one request at a time on one NVIDIA A100 GPU, after the agent ran for 36 hours with no human involvement. The resulting engine reaches 104.9 tokens per second on MT-Bench, against 70.9 for SGLang and 59.0 for vLLM, with similar accuracy on GSM8K and HumanEval.

Open-source code: `https://github.com/aibuildai-inc/aibuildai-llm-inference-agent`

Correspondence: * Pengtao Xie, `pengtao.xie@aibuildai.io`

AIBuildAI

1 Introduction

Training a large language model is a single, enormous expense. Serving it is a small expense repeated without end, and the repetition is what dominates: a model that takes weeks to train answers questions for years. Every improvement in how fast it answers is multiplied by the whole deployed life of the model, which is why inference speed, rather than training speed, is where most of the remaining money and most of the remaining user patience are found. Speed means two different things here, and they pull in different directions. A service with many users at once cares about throughput — total tokens per second across everybody — and can buy it by batching requests together, so one pass over the weights serves many people. A single user waiting for an answer cares about latency, and batching does nothing for them: their tokens still arrive one at a time, as fast as the hardware can produce them one at a time. The second case is the harder one, and it is the common one for what people now build — a chat turn, a coding assistant's completion, one step of an agent that cannot continue until the answer arrives. This report is about that case, where the only way to go faster is to produce each token for less work.

Several established methods make a served model faster, and they are not interchangeable, because they do not all leave the model's answers intact. Using a smaller model, or distilling a big one into a small one, is the fastest option and the least honest: the answers change, and deciding whether they changed acceptably is its own research problem. Quantising the weights to fewer bits is often nearly free in quality and sometimes not, and which it is depends on the model and the task. Serving the model better — fused kernels, paged attention, CUDA graphs, a purpose-built serving framework instead of a training loop — is pure gain with no change to the output, and it is large: measured here, moving from a HuggingFace `generate()` loop to SGLang [4] is worth 4.2×. But it is a one-off, and once you are on a good framework there is no second helping.

Speculative decoding [1], the layer this report demonstrates the agent on, is the one that leaves the weights untouched and, under greedy decoding, leaves the chosen tokens untouched as well — and it composes with the others rather than competing with them. The 4.2× from the framework and the 1.5× from the draft head measured here multiply to 6.2× end to end. That is what singles it out: most speedups ask you to accept a different model, while this one asks you to build something extra and keep the model you already approved. It works because a server generating text for one user at a time leaves the GPU mostly waiting. Producing a single token requires reading every weight in the model out of memory, and reading memory is slow compared with doing arithmetic, so the arithmetic units sit idle while the data arrives — and reading the weights to produce one token costs almost exactly what reading them to produce five would. Speculative decoding spends that idle capacity: a small, fast draft head guesses the next few tokens, and the big model checks all of the guesses in a single pass, the same single read of the weights that would otherwise have produced one token. Guesses matching what the big model would have said are kept; the first wrong guess and everything after it is discarded. When the head guesses well, the server emits three or four tokens for the price of one, and because a guess is kept only if it equals the token the big model would have chosen anyway, the output under greedy decoding is the text that model would have produced on its own.

The difficulty is building it, which means four decisions that cannot be made independently: **how big to make the draft head**, since bigger heads guess better but cost more on every guess; **what text to train it on**, since it has to imitate this particular model on the kind of input it will actually be asked for; **how to train it**, since at serving time it guesses several tokens in a row and feeds its own guesses back to itself, and training has to match that; and how far ahead to guess and **how many alternatives to carry**, which raises both the possible payoff and the cost of each cycle. In the run described here two of those choices flipped sign depending on the others. A head size that was the right call with 5M training tokens was *worse* than a smaller head when only 2.4M were available, and a change to the training procedure that gained 4.3% when the head guessed three tokens ahead gained nothing once it guessed four. The four cannot be tuned one at a time and expected to hold. Each attempt is also expensive, because the head must be trained before it can be timed: only tens of attempts fit a realistic budget, and a bad attempt costs as much as a good one. Pointing a single general coding agent at this works, and it is how most published results of this kind were produced, but a long single session has two habits that hurt here. It settles early on one account of where the bottleneck is, and every later decision is conditioned on that account, so a direction that stops paying tends to be refined rather than abandoned. The second habit is worse and is specific to speed work: timing a GPU is noisy, and the noise runs one way, since an interruption can make a measurement slower but never faster. An agent that measures its own work will accept a 1% improvement that is not there, and can spend a whole budget improving a number that never moved.

Section 3.1 describes the search, how steps follow one another, and the written knowledge the agents read before deciding anything. Section 3.2 describes the score contract: how a step gets its score, and why no agent grades its own work. Section 3.3 describes what carries over between runs: the records a finished run leaves behind are folded into the guides the next run reads, which is why each run starts from better priors than the last. Section 4 reports what the agent produced, timed against three baselines we re-measured ourselves — plain HuggingFace, SGLang [4] and vLLM [5] — and checked for accuracy changes on GSM8K [7] and HumanEval [8]. Section 4.4 walks through all ten steps, including three ideas the agent tried and discarded because its own measurements said no. An inference engine is not one artefact but a stack: kernels, memory management, scheduling, and – the only layer that has to be learned for each model – a speculator. This report demonstrates the agent on that layer, within a run end to end and without human involvement, because it is the layer where the work is research rather than engineering: the choices interact, each one costs hours of GPU time to evaluate, and the answer differs per model. The other layers are future capabilities, not demonstrated ones: nothing here tests the agent on quantisation, on distillation or on serving configuration, so nothing here claims it. What the report does claim about the stack as a whole is measured in Section 4: the learned layer multiplies with the engineered ones rather than competing with them, and the two together take this model from 16.8 to 104.9 tokens per second.

2 Preliminaries

2.1 Problem formulation

The agent is given a served model, a serving framework, a benchmark and a time budget, and must produce a draft head together with the settings that serve it. The quantity being maximised is speedup: tokens per second with the head divided by tokens per second without it, both measured on the same machine in the same session. That speedup factors into exactly two terms,

$$\text{speedup} = \frac{\tau}{c}, \tag{1}$$

where τ is the number of tokens the server gets per check — if the head guesses four tokens and three survive, that cycle contributed three — and c is the cost of one check cycle relative to one ordinary decoding step. Higher τ is better, lower c is better.

The factorisation organises everything that follows, because the two terms are improved by unrelated means. Raising τ means training a better head, which is slow, expensive, and may not pay back inside one step. Lowering c means changing serving settings, which is fast and needs no training, but there are only a few such settings and they run out. A change should therefore be attributed to one term or the other before it is kept: a step that leaves the speedup flat while raising τ has made real progress, and a step that raises the speedup by lowering c has not improved the head at all.

Both τ and c are read off timings, so the attribution just described is only as good as the timings behind it, and speedup itself is a ratio of two of them. How that ratio is taken therefore has to be settled before any number in this report can be read. Two mistakes are easy to make and both inflate the ratio. The first is a stale baseline: a “without speculation” number taken from an earlier session silently absorbs every change in the machine’s state since, so both numbers must come from the same session and the same process. The second is averaging noise that runs one way — an interrupted timing run comes out slower, and nothing makes one come out faster — so the fastest of several runs is the better estimate of what the machine can do, not the mean. Two runs do not help, since the median of two numbers is their average and rejects no outlier; three runs, and report the fastest. Every speedup quoted below is measured this way, the scoring program of Section 3.2 enforces it on each step, and Table 4 gives the resulting noise floor.

2.2 EAGLE draft heads and the verification tree

The draft head used throughout is an EAGLE head [2]: a single transformer layer that works inside the served model's own internal representation. Given that model's internal state after position j and the embedding of the token at position $j + 1$, it predicts what the internal state at $j + 1$ would be, and the served model's own output layer — frozen, shared, never copied into the head — turns that prediction into a token. Because it borrows the vocabulary layers rather than owning them, the head is small: 328M parameters against the served model's 14.8B.

At serving time the head runs on its own predictions to build a small tree of candidate tokens, which the served model then checks in a single pass [3]. Three numbers define that tree: the depth S, how many tokens ahead the head guesses; the branching k, how many alternatives it keeps at each position; and the total number of draft tokens D carried into the check. Depth sets a hard ceiling on τ, since a tree of depth S can contribute at most $S + 1$ tokens per check, and raising it lifts that ceiling at the price of one more run of the head per cycle, which raises c. All three are serving-time settings, so the same trained head can be re-served under a different tree with no retraining. That turns out to matter: one of the ten experiments described in Section 4.4 changed nothing but these three numbers and produced the second-largest gain of the whole run, at no training cost at all.

3 Method

The method has two halves, and the separation between them is the point. One half is a search: a line of ten experiments, each run from start to finish by a fresh agent session, each handing the next one a written report. The other half is a scoring program that the agents do not run and cannot influence. It takes whatever a step delivered, re-times it against a baseline it measures itself, and returns the one number the search is ranked by. The search supplies ideas; the scoring program decides what they were worth. Figure 1 shows the arrangement.

3.1 The search, and what the agents read first

The search runs in a line. Step t is handled by a brand-new agent session with a six-hour budget. It receives the task description, the accumulated code, and *one written report* from step $t - 1$. That report states where the previous step started, what it believed, what it changed, what it measured, what it rejected, and a ranked list of the ideas it did not get to. The agent then runs one complete experiment: form a hypothesis, write and debug the code, build or extend the training set, train the head while watching the loss, time its candidates against the current best, and write its own report. Then the scoring program prices what it banked. Restricting the channel between steps to a written document is deliberate. An agent that cannot hand its successor a conversation has to say what it learned in a form that survives being read cold — which is also the form a person can read. The ten reports from this run are quoted in Appendix C.

Before deciding anything, every agent reads seven written guides. Each one records what was actually measured and the conditions under which it held, rather than general advice. Three of them are about measurement: how to measure — same-session baselines, three runs, keep the fastest, and the noise floor below which a difference should not be believed (Table 4); what a quality claim may say, meaning what follows from the exact check under greedy decoding, what does not, and which statistical test to use; and the speedup formula, Equation (1), with how to tell whether a measured change came from τ or from c. Two are about what to try: a catalogue of head designs, training objectives, ways to build a training set, and tree shapes, each with the conditions under which it has paid off and the conditions under which it has not; and the recipe so far, meaning the head and

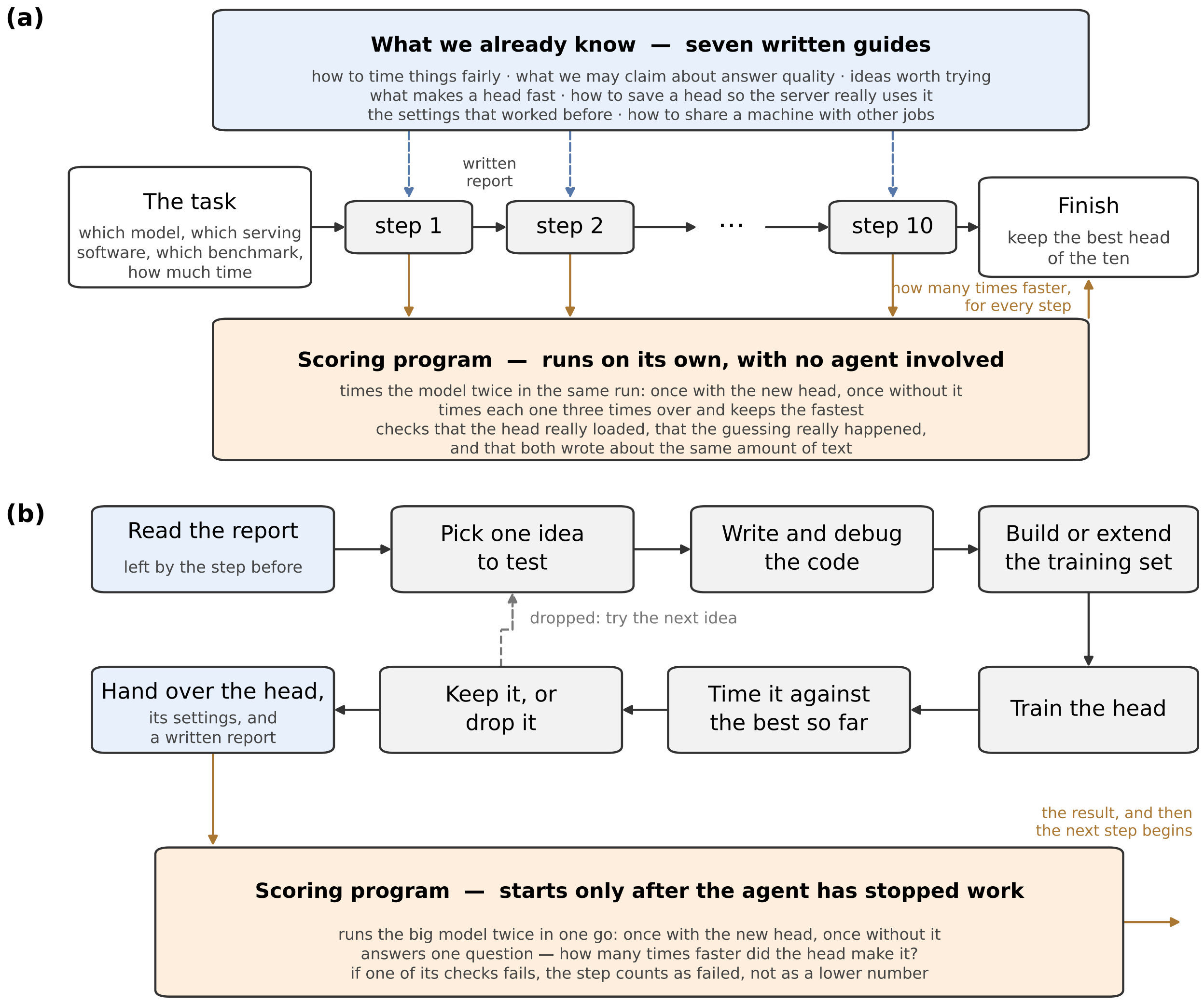
(a)
What we already know — seven written guides
how to time things fairly · what we may claim about answer quality · ideas worth trying
what makes a head fast · how to save a head so the server really uses it
the settings that worked before · how to share a machine with other jobs
The task
which model, which serving software, which benchmark, how much time
written report
step 1
step 2
···
step 10
Finish
keep the best head of the ten
how many times faster, for every step
Scoring program — runs on its own, with no agent involved
times the model twice in the same run: once with the new head, once without it
times each one three times over and keeps the fastest
checks that the head really loaded, that the guessing really happened,
and that both wrote about the same amount of text
(b)
Read the report
left by the step before
Pick one idea to test
Write and debug the code
Build or extend the training set
dropped: try the next idea
Hand over the head,
its settings, and a written report
Keep it, or drop it
Time it against the best so far
Train the head
the result, and then the next step begins
Scoring program — starts only after the agent has stopped work
runs the big model twice in one go: once with the new head, once without it
answers one question — how many times faster did the head make it?
if one of its checks fails, the step counts as failed, not as a lower number

Figure 1 | **How the system works. (a)** The search runs in a line, not a tree. The only thing an agent inherits from the step before is a written report (Appendix C); every agent first reads the same seven guides (Appendix A). **(b)** One step is one complete experiment, run start to finish by a single fresh agent session. The score is not computed by the agent that earned it: a separate program with its own GPUs re-times both the plain and the speculative setup after that session has closed, and returns the speedup and τ (Appendix B). If one of its checks fails, the step is marked failed rather than given a lower score. Not shown: the loop that closes between runs, where records become the guides the next run reads (Section 3.3).

training-set settings that eight earlier searches on other models converged on, each with the data size it was measured at. The last two are about the machinery: how to package a head so the framework really loads it instead of quietly falling back, and how to share a GPU host with other jobs, including how to tell a killed process from one that ran out of memory.

These guides are a starting vocabulary, not a rulebook, and two moments in the Qwen2.5-14B search show the difference. Early on, an agent read the 30 benchmark prompts for itself and found they split evenly between chat, reasoning and code, which contradicted the recipe guide's claim that the prompt set was mostly chat — a claim that had been measured on a different benchmark. That agent wrote the contradiction into its report and planned against what it had just measured rather than against the guide. Later, another agent settled one of the guide's open questions in the negative: making the head wider changed nothing. Both episodes are described with their numbers in Section 4.4.

3.2 The score contract: the agent cannot grade itself

Each step delivers a trained head and its serving settings into a fixed folder layout. Then a *separate* program takes over. It runs in its own process, with its own GPUs, after the agent's session has already closed. It loads the served model twice — once plain, once with the delivered head — times both on the same prompts in the same process, and returns

$$\text{score} \;=\; \frac{\text{fastest of the runs with the head}}{\text{fastest of the runs without it}}.$$

Three things about this arrangement do work that instructions alone would not:

- **The baseline is measured inside the scored run.** Both numbers come from one process on one machine at one moment, so the ratio cannot be inflated by a stale denominator. Across the ten scored steps of that search the baseline stayed within a spread of 0.25%. That stability is what makes the step-to-step differences in Section 4.4 believable.
- **The agent never sees its own score.** An agent may time its own candidates, and it has to, in order to choose between them. But the number that goes into the search is produced afterwards by this program. So if an agent relaxed the protocol to make its own measurement convenient, that shortcut does not reach the official number.
- **The checks are part of the score, not advice.** The program verifies three things, and a failure is a failure, not a lower score. First, that the head really loaded: several serving frameworks skip a weight whose name they do not recognise, print a warning, and carry on, leaving a randomly initialised head that still runs, just slowly. This has to be caught because it does not look like an error. Second, that guessing really happened: the count of check cycles must be above zero, since a head that loads but is never used would otherwise look like a baseline. Third, that the two setups wrote comparable amounts of text, because otherwise a speedup can be manufactured by generating less.

3.3 What improves between runs, not just within one

Two loops run here, and only the inner one is visible in a trajectory plot.

The inner loop is the search of Section 3.1: ten steps, each inheriting a written record, each improving the *head*. It ends when the budget does, and the head it delivers is the result.

The outer loop improves the *agent*. The written guides every step reads before deciding anything are not a fixed rulebook: they are where the system keeps what its own experiments have established, and they are revised from the records a finished run leaves behind. The clearest case in this run is in Section 4.4. A guide said to use a 13824-wide head. A step measured that this holds only above roughly 5M training tokens — an earlier run had adopted the width at 2.37M tokens and scored 1.7 where a narrower head had already reached 2.0 — and the guide was rewritten to state the data requirement first and the width second. A later run reading that guide does not repeat the mistake, and does not spend a step rediscovering it.

That is why the three rows in Table 1 belong in the same table. run1, run2 and run3 are not three attempts at the same thing; each started from guides the previous one had corrected, and they reached 1.2×, 1.4× and 1.5×. The heads got better because the priors did.

Two limits are worth stating plainly, because the loop is easy to overclaim. First, the agent does not modify its own code, its scoring program, or the search procedure: what it revises is the knowledge it reads, which is text.

Table 1 | **Speed and accuracy.** Every row serves the same Qwen2.5-14B-Instruct weights; all are greedy, one request at a time, on one A100-80GB. *tok/s* is MT-Bench at temperature 0. *Speedup* compares each row with the default SGLang run from the same session. GSM8K is the first 500 problems, HumanEval all 164. The vLLM row uses vLLM's own defaults. *run1* and *run2* are earlier AIBuildAI searches on the same task, shown so the progress across runs is visible; the last row is the run this report describes. All three baseline frameworks agree on accuracy within noise: paired against default SGLang on the same problems.

Setup	tok/s	Speedup	GSM8K	HumanEval
Qwen2.5-14B, plain HuggingFace loop, eager	16.8	0.3×	91.8	78.7
vLLM	59.0	0.8×	92.2	79.9
SGLang	70.9	1.0×	**92.2**	**80.5**
AIBuildAI [9] (run1)	87.6	1.2×	91.2	79.9
AIBuildAI [9] (run2)	100.1	1.4×	90.8	79.3
AIBuildAI [9] (run3, this report)	**104.9**	**1.5×**	**92.2**	**80.5**

Second, that revision is currently applied between runs by the team, from the records — the agents produce the measurements and the contradictions, a human decides what the guide should now say. Closing that last step is the obvious next piece of work, and it is a small one: the records already state what changed, what was measured, and under what conditions it held, which is exactly what a guide entry needs.

4 Results

4.1 Setup

The big model is Qwen2.5-14B-Instruct in bfloat16, served by SGLang on one NVIDIA A100-SXM4-80GB, one request at a time, greedy decoding. The search ran with no human involvement under a 48-hour limit and used 36 hours, with a $250 spending cap. Every agent role ran on Claude Opus 4.8, except the initial setup agent, which ran on Claude Sonnet 5. During the search, scores come from a fixed set of 30 prompts at 128 new tokens. That short benchmark reads about 10% high compared with MT-Bench [6], so the two are related below rather than mixed: the search's final 2.2× corresponds to the 1.5× we report.

Everything in this section was re-measured *outside* the search, by an evaluation harness the search never saw. The headline benchmark is MT-Bench: 80 questions, two turns each, up to 1024 new tokens per turn. Three baselines were re-measured on that exact protocol, each in the same session as the setup it is compared against:

- The plain HuggingFace generation loop in eager mode, deliberately the weak reference, included to show how much of any headline number is just the serving framework.
- Default SGLang running its own ngram speculator, which is the honest baseline.
- Default vLLM running its own ngram speculator, a second and independent production framework, included to check that our baseline is not simply an unspeculated one.

Quality score was reported as accuracy that measured on GSM8K [7] (first 500 test problems, graded by whether the final number is right) and HumanEval [8] (all 164, graded by running the unit tests).

Table 2 | **The speedup depends on the task.** The same head, the same machine, four different workloads, with the speculative timing re-measured in the same session for each row. Step-by-step arithmetic is the most predictable text, so the head guesses right for longer; open-ended chat at temperature 1.0 is the least predictable. A speedup number for a draft head is therefore meaningless without naming the benchmark. The plain column is each framework unspeculated, so this table isolates how the workload changes the head's gain; n-gram speculation was measured only on MT-Bench, so it is not a column here.

Workload	with head	Speedup	τ
GSM8K (500 problems)	142.6	4.4	
MT-Bench, T=0	104.9	3.2	
HumanEval (164 problems)	87.8	3.8	
MT-Bench, T=1	93.89	2.9	

Table 3 | **Accuracy is unchanged.** Each problem was solved twice — once plain, once with the draft head — in the same session, at temperature 0, so the two can be compared problem by problem. We use the paired (McNemar) test because two equal percentages can mean either identical answers or many changes that happen to cancel out; only the problems where the two disagree carry information. *win:lose* counts problems the run with the head got right and the plain run got wrong, against the reverse.

Benchmark	plain	with head	change	disagreements (win:lose)
GSM8K (n = 500)	461/500 = 92.2%	461/500 = 92.2%	+0.0	5 : 5
HumanEval (n = 164)	132/164 = 80.5%	132/164 = 80.5%	+0.0	2 : 2
Combined (n = 664)			+0.00	**7 : 7**, p = 1.000

4.2 How much faster

Table 1 is the summary. The head the agent produced serves MT-Bench at 104.9 tokens per second where the same framework's own speculator manages 70.9 — a factor of 1.5 — and gets 3.2 tokens per check cycle.

Against the plain HuggingFace loop the total factor is **6.2**×, and it is worth splitting that in two, because only half of it is ours:

$$\underbrace{4.2\times}_{\text{switching to a real serving framework}} \times \underbrace{1.5\times}_{\text{our draft head}} = 6.2\times.$$

Read the other way, that decomposition says what an inference engine is made of. The 4.2× is engineering that is written once and applies to every model: kernels, paged attention, CUDA graphs, a scheduler. The 1.5× speedup (compared with default SGLang) is the layer that cannot be written once, because it has to be fitted to this model's own output distribution — and it is the layer the agent produced, from a task description, with no human in the loop. The delivered artefact is a drop-in part of the engine: the head's weights plus the serving settings that run them, which SGLang loads directly. At temperature 1.0 the same head gives 93.9 tok/s, τ = 2.9.

Table 2 shows why the benchmark has to be named alongside the number. τ from 2.9 to 4.4, depending only on what it is asked to write. Quoting the GSM8K figure as "our speedup" would overstate it by 35% relative to MT-Bench.

Table 4 | **How small a difference we can actually measure.** Established before believing any improvement. A reported gain below roughly 1.5% is inside the measurement noise and is not evidence of anything; the evaluation harness states its reproduction tolerance as ±2%.

What is being compared	Spread
The same head, timed twice in the same process	~0.3%
The same head, agent's own timing against the scoring program	~1%
The speculative setup, run to run	~1.2%

4.3 Accuracy did not change

Out of 664 graded problems, 7 changed in each direction (Table 3). The McNemar exact test gives $p = 1.000$. Changes in both directions in equal numbers is what random noise looks like. If the changes had all gone one way — the draft head losing problems the plain run got right — that would be the warning sign. The two runs' output lengths stayed within 2.1% of each other, so the comparison is not distorted by one of them simply writing less.

It is worth being precise about what this does and does not prove, because the obvious stronger claim is false:

- **What is true.** Under greedy decoding the check keeps a guessed token only if it equals the big model's own top choice, and stores the big model's token either way. The draft head therefore cannot change which token gets chosen.
- **What is not true.** The output is *not* bit-for-bit identical. Floating-point ties round differently between a one-token-at-a-time pass and a batched check of a whole tree, so the text can drift. On a 30-prompt test only 10 to 15 of the 30 outputs matched the plain run exactly, with about 0.86% of tokens different — while the plain run reproduces itself 30 times out of 30. So speculation is the cause of the drift.

The fair statement is: the big model's own choices are preserved, the exact text is not guaranteed identical, and here is the count. We do not claim losslessness. The same test across frameworks gives the same answer. default vLLM against default SGLang, same problems, disagrees 6:6 on GSM8K and 1:0 on HumanEval, $p = 1.000$ in both. The two production frameworks are statistically indistinguishable on accuracy, which is what makes the vLLM baseline in Table 1 usable as a cross-check.

4.4 The ten steps

Figure 2 and Table 5 trace the run. Step 1 had to build everything: the shared code folder contained a fine-tuning template for a different model on a different task, with no draft pipeline and no head at all. So the first step wrote the head, the data generator, the trainer and the packaging from scratch, and reached 1.3×. The nine steps after it alternate between the two halves of Equation (1), and the alternation is not an accident: the agent worked out which half each change had moved before keeping it. There are four passages are worth drawing out:

- **Step 3: train the head on what the big model actually says (+10.3%).** Steps 1 and 2 trained the head on human-written answers from public datasets. But at serving time the head is always looking at text the served model itself produced. The agent spotted the mismatch, regenerated the training set by having the big model write the answers itself, and held the head size, the tree, and every training setting fixed so the step tested only that one change.
- **Step 5: more data beats a bigger head (+13.7%).** Growing the training set from 2.37M to 4.37M tokens

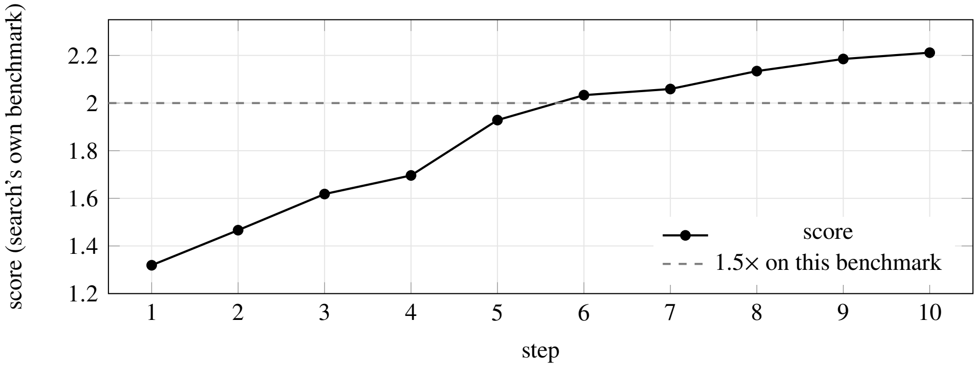

Figure 2 | **The search, step by step.** The score rises at every step, with the smallest gains at steps 6–7, and it never flattens out: the last step still gained 1.2%. The run ended because the time budget ran out, not because the method stopped improving — so the head we shipped is a floor on this recipe. This short benchmark reads about 10% high; its final 2.2 is the 1.5× of Table 1.

Table 5 | **What moved the number at each step.** *Moved* says which half of Equation (1) the step attacked: τ (better guesses, needs training) or c (cheaper cycles, a serving setting that needs no training). Scores are the scoring program's, on the search's own short benchmark. Training-set sizes are in tokens of text the big model generated itself.

Step	What it changed	Moved	Where it ended up	Score	τ
1	Build everything from scratch	—	head 8192 wide, 1.1M tokens, tree 4/16/64	1.3	2.2
2	Wider head (13824); rebalance data	τ	13824 wide, 2.1M tokens	1.5	2.5
3	Train on the big model's own output	τ	2.3M self-generated tokens	1.6	2.8
4	Match data to the benchmark; retune tree	c	tree 3/24/64	1.7	2.7
5	More data (2.37M→4.37M), 4 epochs	τ	chat-heavy 8000/3500/3500	1.9	3.2
6	Train on its own guesses, 3 deep	τ	same data, new training loss	2.0	3.3
7	+6000 chat prompts from a 2nd source	τ	5.94M tokens, chat 14000	2.1	3.4
8	Guess 4 ahead instead of 3 (no training)	c	tree 4/24/64	2.1	3.7
9	Add reasoning and code data	τ	chat 14000 / reas. 8500 / code 8500	2.2	3.8
10	Add more of both	τ	chat 14000 / reas. 11500 / code 11500	**2.2**	**3.8**

— with the new text concentrated on the weakest topic — and training four epochs instead of two was the single largest gain of the run: τ went 2.7 → 3.2 and the speedup 1.7 → 1.9, measured side by side against the previous best with identical baselines.

The same step also retested making the head wider, which the recipe guide had left open. Going from 13824 to 16384 gave 1.8 against 1.8 — the same number, given that repeated timings of one head vary by 0.30%. Width does nothing, now measured twice.

This is also where earlier advice turned out to have a hidden condition. "Use a 13824-wide head" only holds *if you have at least about 5M training tokens*. An earlier run adopted that width while keeping a 2.37M-token training set and scored 1.7, where the narrower head had already reached 2.0 . So the guide was rewritten to state the data requirement first and the width second.

- **Steps 6 and 8: fix the training, then change a setting.** Step 6 changed how the head was trained, not what it was trained on. The trainer had only ever shown the head *correct* inputs, but at serving time the head guesses three tokens in a row, feeding its own guesses back in — so after the first guess it is working from its own possibly-wrong output. Training it on its own guesses, at the same depth the server uses, moved τ from 3.2 to 3.3 and carried the run past 2.0× for the first time.

  Step 8 then noticed that the head's τ of 3.4 was already 84% of the hard ceiling for a tree that guesses 3 ahead (which can contribute at most 4). The only way to raise that ceiling is to guess further ahead — and that is a serving setting, not a training change. Sweeping it on the *same weights*, with no retraining at all, moved the speedup from 2.0 to 2.1 and τ from 3.4 to 3.7. The second-largest gain of the whole run cost no GPU training time.

- **Three ideas thrown away on the evidence.** What the run gave up on is as informative as what it kept. In step 5, training for five epochs instead of four flattened out: τ went 3.2 to 3.2 but the speedup went 1.9 to 1.9, lower, and inside the noise either way, so four epochs stayed. In step 6, the one free knob of the new training loss was tried at a second value and measured reliably worse, so the original value stayed. In step 8, having found that guessing four ahead paid off, the agent did the natural follow-up and retrained the head to match that depth; it came out worse than the head trained for depth three (3.6 against 3.7) and was not shipped.
  In each case the rejected idea had a sensible mechanism behind it, and the measurement said no anyway. The data-mix lever was still paying when the budget ran out, but only just: its gain in τ fell from +2.8% at step 9 to +1.0% at step 10 as the reasoning and code slices grew from 3500 to 8500 to 11500 prompts.

5 What comes next

The agent demonstrated here builds one layer of an inference engine. The machinery around it — a score contract the agent cannot reach, a written channel between steps, guides that accumulate what was measured — is not specific to draft heads, and the layers it has not been pointed at are the obvious next capabilities: a quantisation choice is a search over configurations with a quality constraint, and serving configuration is a search over settings with no training at all, which would make each step minutes rather than hours. Neither is claimed here. What this run establishes is that the loop works on the hardest of the three, where a single candidate costs hours of GPU time and the choices interact.

Does the recipe work on much larger models? It was found at 14B on a single GPU. Whether it carries over is a question about the recipe rather than about the agent, and it is the obvious next thing to test: a target large enough to need tensor parallelism changes what the engine does per token, so the tree settings and the acceptance the head can reach may both move. Work on a larger target is under way and will be reported separately.

What we can say already, from preparing that work, is that the binding difficulty changes with scale, and not in the direction we expected. At 14B a bad idea costs one cheap step and the search routes around it. On a target that needs several GPUs, the failures that cost real budget were not about speculative decoding at all: they were about whether a work unit was actually given the hardware it needed, and about how long a framework waits for a multi-GPU engine to shut down. Each of those is invisible while the search looks healthy — steps complete, records get written — and each makes every score meaningless until it is found.

The lesson we take into that work is that the score contract of Section 3.2 has an analogue we had not built. It checks the claims an agent makes about speed. Nothing checked the claims the *platform* makes about resources, and at this scale that is where the expensive mistakes live. A run should verify that it got the GPUs it asked for before it spends a step proving it did not.

Make scoring cheap enough to search properly. The search runs in a line, one step after another, because scoring is expensive: it means loading the big model twice and timing both. That caps a step at 150 minutes and a 48-hour run at ten steps. Most of that cost buys nothing a cheap approximation could not: how often the head guesses the very next token correctly tracks τ well enough to screen candidates, and the step reports show agents screening that way before paying for a full timing run. A two-tier scheme — a cheap screen for choosing, an expensive confirmation for the official number — would allow a branching search on the same budget.

There is one trap here, already visible in this run. In step 5 the cheap screening score fell at every epoch while the official τ rose at every epoch. A cheap metric promoted to the official score would have picked the wrong head.

Write down search strategy, not just training knowledge. The seven guides say what to train and how to measure it, but nothing about how to organise the search itself. The run's cheapest win (step 8: change a serving setting, no training, +4.6%) and its most expensive dead end (step 5's width retest) are both the kind of thing a guide could state up front as a strategy — *sweep the free serving settings on the head you already have before training anything new* — instead of something every run has to rediscover.

Data availability

MT-Bench, GSM8K and HumanEval are public. The training sets were generated by the big model itself, writing answers to public instruction prompts; the source datasets, their licences and the exact filters applied accompany this report, as do all the result files behind Tables 1–4, including the per-problem grading records used for the paired tests.

Code availability

The evaluation harness, the scoring program, the training pipeline, the recipe and the trained head accompany this report can be find in `https://github.com/aibuildai-inc/aibuildai-llm-inference-agent` under the MIT licence.

References

[1] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. *ICML*, 2023.

[2] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. *ICML*, 2024.

[3] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. *EMNLP*, 2024.

[4] Lianmin Zheng et al. SGLang: Efficient execution of structured language model programs. *NeurIPS*, 2024.

[5] Woosuk Kwon et al. Efficient memory management for large language model serving with PagedAttention. *SOSP*, 2023.

[6] Lianmin Zheng et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *NeurIPS Datasets and Benchmarks*, 2023.

[7] Mirzadeh, Iman, et al. Gsm-symbolic: Understanding the limitations of mathematical reasoning in large language models. *International Conference on Learning Representations*, 2025.

[8] Chen, Mark, et al. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374, 2021.

[9] Peijia Qin, Ruiyi Zhang, Qi Cao, Han Guo, Li Zhang, and Pengtao Xie. AIBuildAI-2.5: Efficient autonomous AI model development through LLM-guided tree search. Manuscript, 2026.

A Two of the seven guides

Section 3.1 describes the seven written guides the agents read. Two short excerpts follow. Notice that each one records a measurement and the conditions it held under, rather than giving advice.

A.1 How to measure: the noise floor

Guide: measuring speculative decoding — rule 3.

The judgment Work out what your measurement can resolve before deciding what changed.

The protocol Time both setups in one process, on one machine, in one session. Do three runs, not two: the median of two numbers is just their average, so a second run cannot reject an outlier. Report the *fastest* run, not the average — an interruption can only make a run slower.

The floor Same head, same process: about 0.30%. Same head, agent's own timing against the scoring program: about 1%. The speculative setup, run to run: about 1.2%. Treat anything under about 1.5% as unresolved.

A case A small experiment on tree settings found changes of +2.7% and −0.8% and was written down as "the tree hardly matters". Re-measured on a different head, the tree was worth 5.7%. The original finding was not wrong; it was true only for a head it forgot to name.

A.2 The recipe: data comes before head size

Guide: draft head recipe — head size.

The requirement A 13824-wide head needs at least about 5M tokens of self-generated training text. Below about 3M tokens, use 8192 instead. This is a requirement, not a footnote.

Why it is written that way Given "13824 wide, trained on 5.9M tokens" as one recommendation, a run copied the width and kept a 2.37M-token training set. It scored 1.7, where the narrower head had already reached 2.0.

What was measured Head size and tree held fixed, only the training set changed: 2.37M tokens / 2 epochs → 1.7 (τ 2.7); 4.37M / 2 → 1.8 (3.0); 4.37M / 3 → 1.9 (3.1); 4.37M / 4 → 1.9 (3.2). That is +15.4% from more data and more epochs alone, with no change to the head.

On width 13824 → 16384 on the same 4.37M training set: 1.8 against 1.8 — the same number. Grow the data first. Widen last, if at all, and say what data size you widened at.

B The scoring program

The arrangement described in Section 3.2 is one function the scoring program implements and the search calls. Shortened for readability:

```
def score(output_dir: Path) -> dict:
    """Price whatever the agent left in output_dir. Runs in its own
    process, with its own GPUs, after the agent’s session has closed."""

    head, settings = load_delivered(output_dir)   # fails loudly, never silently

    # Check 1: the framework must map every delivered weight.
    # Several frameworks skip a weight whose name they do not recognise,
    # print a warning, and keep going -- leaving a randomly initialised
    # head that still runs. That does not look like an error.
    require_all_weights_mapped(head)

    with engine(big_model, speculation=None) as plain:
        plain_tps = [time_one_run(plain, prompts) for _ in range(n_runs)]

    with engine(big_model, speculation=head, **settings) as method:
        method_tps, checks, tokens = [], 0, 0
        for _ in range(n_runs):
            r = time_one_run(method, prompts)
            method_tps.append(r.tps)
            # The count of check cycles lives in the PER-REQUEST metadata.
            # Reading it from the server info block returns zero, which
            # would silently score every step at zero.
            checks += sum(x.meta_info["spec_verify_ct"] for x in r.requests)
            tokens += sum(x.meta_info["completion_tokens"] for x in r.requests)

    # Check 2: guessing must actually have happened.
    if checks == 0:
        return failure("the head never got used")

    # Check 3: both setups must have written comparable amounts of text.
    require_token_parity(plain, method, tolerance=0.05)

    return {"score": max(method_tps) / max(plain_tps),
            "tau": tokens / checks,
            "plain_tps": max(plain_tps), "method_tps": max(method_tps)}
```

Check 1 and the comment about where the cycle count lives are both there because of real failures. A framework that silently skips weights it does not recognise will happily serve a randomly initialised head at about 0.84× — slower than no speculation at all, but not obviously broken. And reading the cycle count from the wrong place returns zero, which would have scored every single step of a run at zero.

C The ten step reports

Each step leaves a written report, and that report is the only thing the next step inherits. Here is the headline of each, in the words of the agent that wrote it.

1. **Build a draft head for Qwen2.5-14B-Instruct in SGLang.** “There was no EAGLE pipeline and no draft head at all — a baseline score here is about 1.0. So step 1 had to build the whole pipeline from the task contract, not tune an existing one.”
2. **Wider head, more and better-balanced data.** “I read the 30 scored prompts directly. They split cleanly: 0–9 chat, 10–19 maths and reasoning, 20–29 code — a third each. This contradicts the recipe guide’s claim that this task’s prompts are mostly chat (that was measured on a different benchmark).”
3. **Train on the big model’s own output.** “SGLang drafts over the big model’s own output, but steps 1–2 trained on human-written text — a mismatch between training and serving. I took that lever.”
4. **Match the data to the benchmark, then retune the tree.** “3/24/64 beats 4/16/64 by +3.5% (confirmed twice in the same session) by cutting one guessing step per cycle. The tree retune was the larger lever.”

5. **More self-generated data (chat-heavy) and more epochs.** "τ 2.7 → 3.0 (+10.5%), speedup 1.7 → 1.8 (+9.4%), with no overlap between the runs and identical baselines. . . . A 5-epoch probe then flattened out, so epochs saturate at 4–5."
6. **Train the head on its own guesses.** "The trainer was purely single-step: the head only ever saw the *true* input, yet SGLang makes it guess 3 deep, feeding it its *own* prediction from the second guess on. . . . the first time this project crosses 2.0×."
7. **Add data for the weakest topic.** "Chat is by far the weakest topic (the head gets it right 29% of the time, against 56% for reasoning and 50% for code), and chat training was stuck at one dataset's 8000-row ceiling."
8. **Guess 4 ahead instead of 3.** "The step-7 head was limited by depth, not accuracy. . . . How far ahead to guess is a serving setting — the *same weights* run deeper — so I swept it with no retraining. . . . I also retrained the head to match the deeper tree; it measured worse, so it is not shipped."
9. **Rebalance the training data toward the benchmark's thirds.** "Earlier steps kept pouring data into chat because chat looked weakest, and chat saturated. But the score is the equal-weight average over the three topics, so the next training token belongs wherever it buys the most."
10. **Push the balance further, adding only (removing nothing).** "Second improvement in a row from data balance; the lever is now close to saturation — the gain fell from +2.8% to +1.0% as the reasoning and code slices grew from 3500 to 8500 to 11500."